

LensaRak — System Guide

For the team — how it runs, how it works, what it can and can't do yet

LensaRak is a camera-based inventory system for Petrosains' storeroom, built for the AI Innovators Challenge 2026. It watches a camera feed, recognises items with a trained YOLOv8 model, and updates a local stock ledger — automatically when it's confident, and only after a human confirms when it isn't.

1. How to Run It

Everything is launched from **LensaRak.bat** in the project folder — double-click it, no typing commands needed. It opens a simple menu:

Option	What it does
1. Check-out items	Webcam, live preview. Use when items are being taken OUT of the store.
2. Check-out items (phone)	Same as above but the camera source is a phone running an IP-camera app instead of the laptop webcam.
3. Bulk return	Webcam, live preview. Use when a batch of mixed items is being returned at once — this is the main demo scenario.
4. Bulk return (phone)	Same as option 3 but via phone camera. Both phone options ask for a rotation (90/180/270) if the preview looks sideways — purely a display fix, no effect on the network connection.
5. Review pending items	No camera needed. Opens the console for clearing anything the system flagged for a human to check.
6. View transaction history	No camera needed. Read-only — every transaction ever logged, with filters for pending/reviewed/by item.
7. Export full ledger	No camera needed. Writes the full ledger and current stock to two .csv files that open directly in Excel — clearer than reading the cmd window.

Advanced / manual: each mode can also be run directly, e.g. `python lensarak_vision.py --source webcam --mode bulk_return --live` — see the top of `lensarak_vision.py` for every flag.

2. Keyboard Controls

During a live scan (after pressing SPACE to capture a frame):

Key	What it does
SPACE	Capture the current frame and run detection on it.
ENTER	Accept the suggested quantity for the item currently shown (either the camera's own box count, or an OCR-read number off a packaging label) and move to the next item.

U	Switch to typing the real quantity by hand — use this when the suggested number is wrong, or there's no printed label to read at all (e.g. a bagged item). Type digits, Backspace to correct, ENTER to save.
Q	Quit the live preview window.
(no key)	After ~4 seconds with no response, the suggested quantity is auto-confirmed — same as pressing ENTER — so one distracted worker never blocks the line.

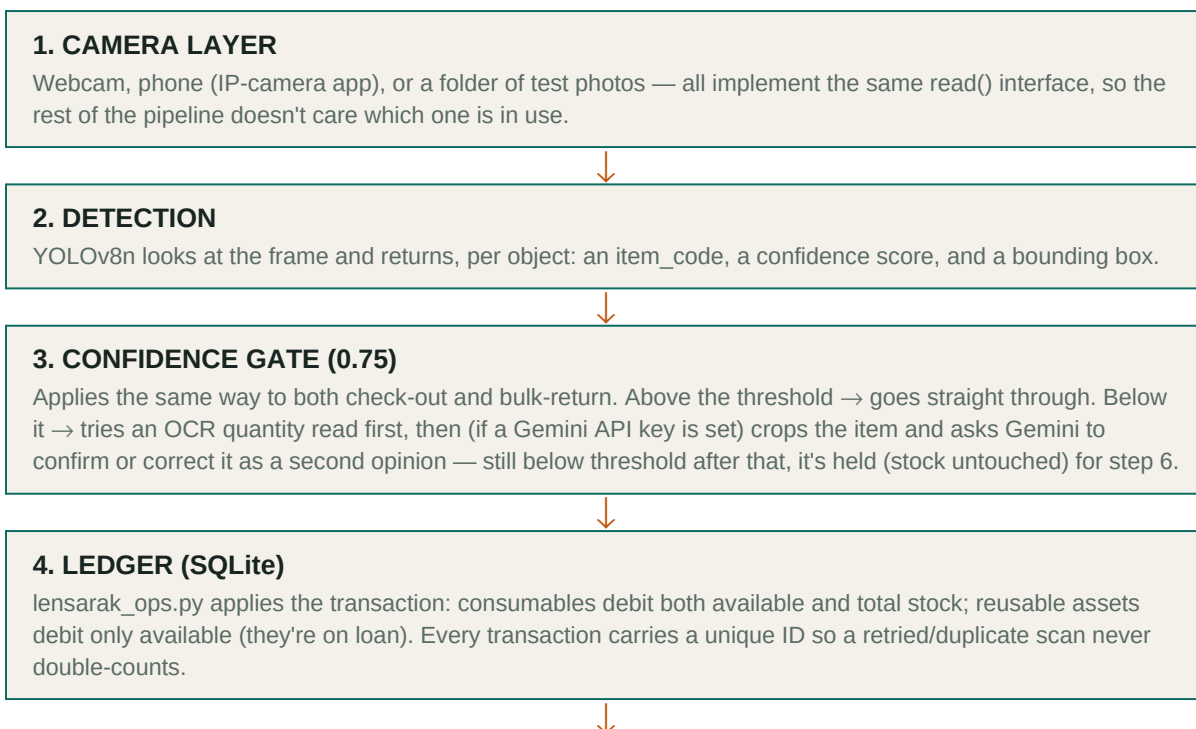
In the review console (`lensarak_review.py` — for anything flagged `needs_review`):

Key	What it does
ENTER	The camera got it right — approve exactly as recorded, stock updates now.
U	Item is correct but the quantity is wrong — type the real quantity.
C	The item itself was misidentified — type the correct <code>item_code</code> (or 'list' to see the catalogue), then confirm/edit the quantity the same way.
N	Reject — nothing is applied, stock stays untouched.

Important: a **C** correction never overwrites what the camera originally read — that record stays, marked as reviewed and corrected. A brand-new transaction is created for the real item, so there's always a full trail of both what the camera thought and what a human confirmed.

3. System Architecture

The pipeline, start to finish:



5. NEEDS_REVIEW QUEUE

Anything still not confident enough is recorded but held — stock is NOT touched until a human clears it.



6. REVIEW CONSOLE

lensarak_review.py — a human approves, corrects the quantity, corrects the item, or rejects. This is where stock for held items finally moves.

Core principle: the system never silently guesses. Confident reads apply automatically; anything else waits for a human — and every correction keeps both the original camera read and the human's fix on record.

4. What It Does Well

- **Camera-agnostic** — webcam, phone-as-IP-camera, or a test-photo folder all run the exact same pipeline, no code changes.
- **Offline-capable** — once trained, everything (detection, OCR, ledger) runs with no internet. Gemini second-opinion is the one optional exception, and it's off by default.
- **Confidence-gated, never silent** — low-confidence reads are held for review instead of guessed, and corrections never erase the original detection.
- **Packaging-aware** — OCR reads printed quantity labels off bags/boxes automatically where present; packaging-to-base-unit conversion (e.g. 1 pack = 40 units) is handled centrally.
- **Idempotent** — every transaction has a unique ID, so a retried or duplicate scan can never double-deduct stock.

5. Current Limitations (be upfront about these)

- **Dataset coverage is still growing** — not all 109 catalogue items have enough training photos yet; under-trained items will have low confidence and route to review more often.
- **Bagged/boxed items without a printed label** — the model can't see through opaque packaging, so for items we haven't photographed inside their packaging, we show one exposed unit to the camera and enter the real quantity by hand (U). This is a deliberate design choice, not a bug.
- **CPU-only inference** — detection runs fine live, but full training runs are slow on a laptop CPU (hours, not minutes) for a large dataset.
- **Gemini second-opinion needs internet** — it's an optional accuracy boost, not a dependency; the system works fully offline without it, just with more items going to manual review.

6. File Reference

File	Purpose
LensaRak.bat	Menu launcher — the normal way to start anything.
lensarak_vision.py	Camera → detection → ledger. Live or single-shot, check-out or bulk-return.
lensarak_ops.py	Core ledger logic: check-out, bulk-return, the confidence gate, packaging conversion, review approval/correction.
lensarak_review.py	Console for clearing the needs_review queue.

lensarak_history.py	Read-only viewer for every transaction ever logged (not just pending review).
lensarak_export.py	Exports the full ledger and current stock to .csv files for Excel.
lensarak_train.py	Pulls a labelled dataset from Roboflow and trains a new best.pt.
lensarak_inventory.db	The actual SQLite database — stock levels and full transaction history.
best.pt	The trained YOLO model currently in use. Vision script picks this up automatically once it exists.